

Zelo+

Guia didático do firmware

Dispenser inteligente de medicamentos com 3 compartimentos
ESP32 · servos · LCD I2C · Telegram · histórico de doses

Funcionamento explicado passo a passo,
com o trecho de código de cada etapa

Projeto acadêmico Zelo+ · Setembro de 2026

Material pedagógico que explica, **passo a passo e na ordem em que as coisas acontecem**, como o dispenser Zelo+ funciona. Em cada etapa aparece só o **trecho de código** responsável por ela (o arquivo completo está em [src/main.cpp](#)).

Sumário

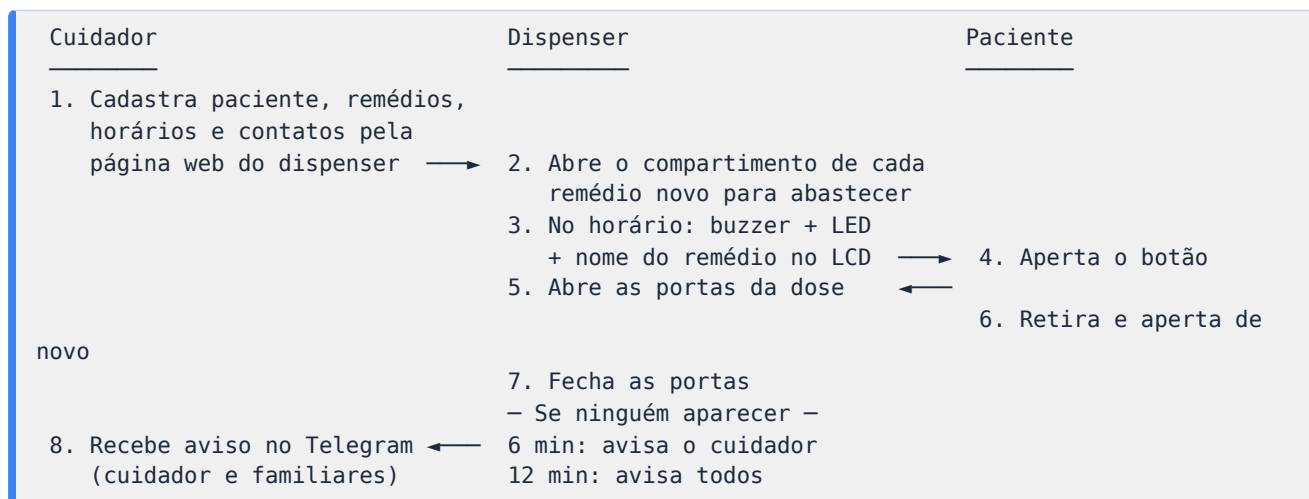
- Parte A — Entendendo o projeto
 - A1. O que é o Zelo+
 - A2. Materiais e ligações
 - A3. Ambiente de desenvolvimento
 - A4. Cinco conceitos antes de começar
- Parte B — O código, passo a passo
 - Passo 1 — Bibliotecas e pinos
 - Passo 2 — Estruturas de dados
 - Passo 3 — `setup()` : ligando tudo
 - Passo 4 — Wi-Fi sem senha no código (captive portal)
 - Passo 5 — Modo normal: relógio pela internet e rotas da página
 - Passo 6 — Memória permanente (Preferences)
 - Passo 7 — A página de cadastro
 - Passo 8 — Salvando o cadastro
 - Passo 9 — Movendo as portas (servos)
 - Passo 10 — O coração: `loop()` e a máquina de estados
 - Passo 11 — Conferindo os horários
 - Passo 12 — O alarme tocando
 - Passo 13 — O paciente aperta o botão
 - Passo 14 — Abastecer, repor e esvaziar
 - Passo 15 — Avisos pelo Telegram
 - Passo 16 — Histórico de doses
 - Passo 17 — Escrevendo no LCD
 - Passo 18 — Modo de teste
- Parte C — Juntando tudo
 - C1. Linha do tempo de uma dose
 - C2. Referência rápida
 - C3. Para praticar

Parte A — Entendendo o projeto

A1. O que é o Zelo+

O Zelo+ é um **dispenser automático de medicamentos** para pacientes (em especial idosos). Ele tem **3 compartimentos**, cada um com um remédio diferente e com uma porta movida por um servo motor.

Rotina do dia a dia:



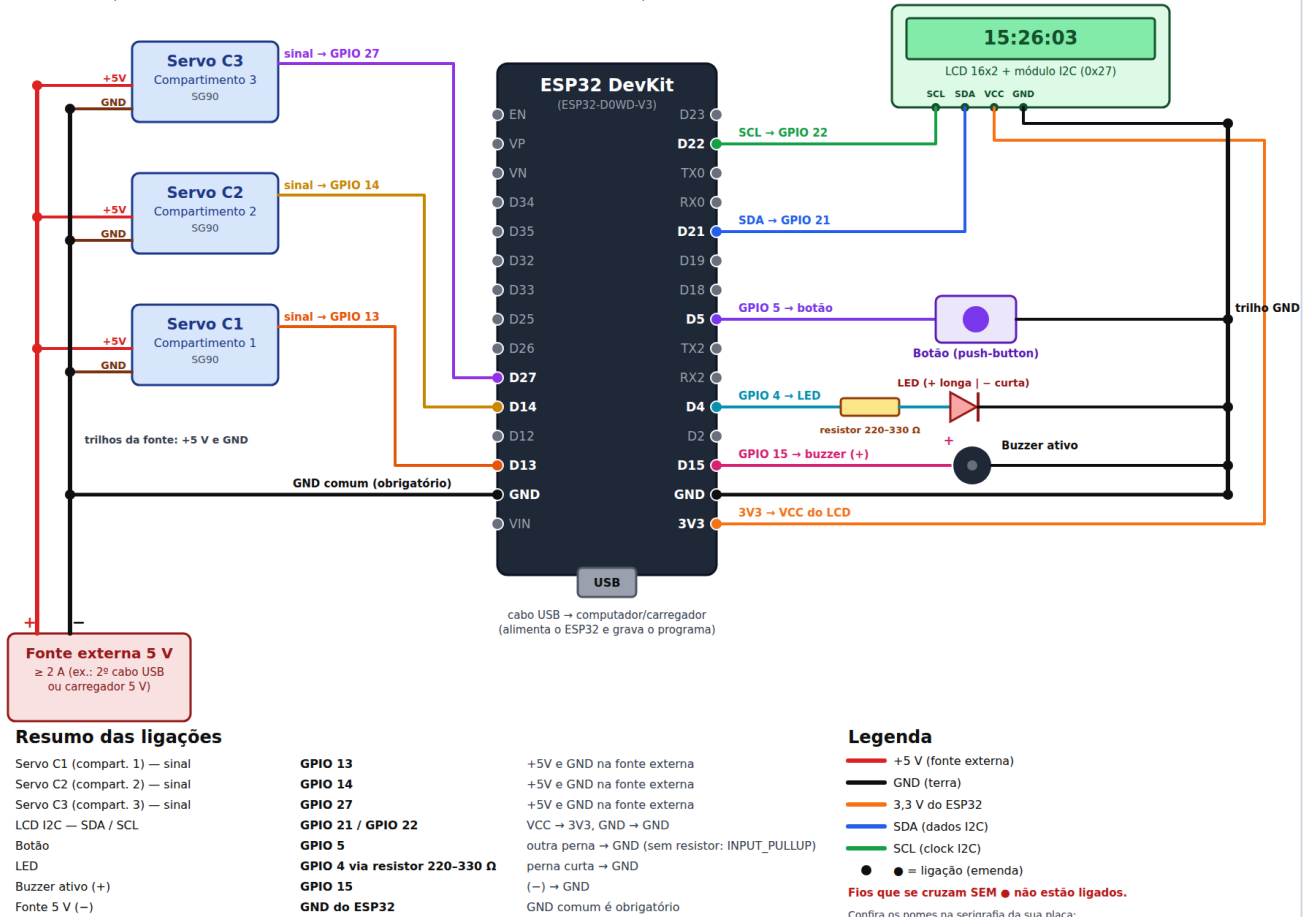
Tudo o que o dispenser precisa saber (remédios, horários, contatos, histórico) fica guardado na **memória interna** da placa e sobrevive a quedas de energia.

A2. Materiais e ligações

Componente	Quantidade	Observação
ESP32 DevKit (30 pinos)	1	O "cérebro". Tem Wi-Fi embutido.
Servo motor SG90	3	Um por compartimento (porta).
LCD 16x2 com módulo I2C (PCF8574, endereço 0x27)	1	Mostra relógio e mensagens.
Buzzer ativo	1	Apita sozinho quando recebe energia.
LED + resistor 220–330 Ω	1	Pisca junto com o buzzer.
Botão (push-button)	1	O paciente aperta para abrir/fechar.
Fonte externa 5 V (≥ 2 A)	1	Alimenta os servos.
Protoboard e jumpers	—	Ligações.

Zelo+ — Diagrama de ligações (ESP32 DevKit 30 pinos)

3 servos (compartimentos) · LCD 16x2 I2C · buzzer ativo · LED · botão · fonte externa 5 V para os servos



(versão vetorial, que pode ser ampliada sem perder qualidade: <docs/ligacoes.svg>)

Ligação	Pino do ESP32	Detalhe
Servo compartimento 1 (sinal)	GPIO 13	+5 V e GND na fonte externa
Servo compartimento 2 (sinal)	GPIO 14	+5 V e GND na fonte externa
Servo compartimento 3 (sinal)	GPIO 27	+5 V e GND na fonte externa
LCD — SDA / SCL	GPIO 21 / GPIO 22	VCC → 3V3, GND → GND
Botão	GPIO 5	outra perna → GND (sem resistor)
LED	GPIO 4	via resistor; perna curta → GND
Buzzer (+)	GPIO 15	(—) → GND
Fonte externa (—)	GND do ESP32	GND comum obrigatório

Por que uma fonte separada para os servos? Um servo puxa muita corrente no momento em que começa a girar. Se ele for ligado no 3V3 do ESP32, a tensão cai e a placa reinicia. Por isso os servos usam uma fonte de 5 V própria, e o **GND da fonte é ligado ao GND do ESP32**: sem essa referência comum, o sinal do pino não é entendido pelo servo.

Por que o GPIO 12 não foi usado? Ele é um pino de "configuração de inicialização" do ESP32; se estiver com algo ligado na hora de ligar a placa, ela pode não iniciar.

A3. Ambiente de desenvolvimento

- **VSCode** com a extensão **PlatformIO**.
- Placa `esp32dev`, framework **Arduino** (a mesma linguagem do Arduino, em C++).

O arquivo `platformio.ini` diz ao PlatformIO qual placa usar e quais bibliotecas baixar:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
upload_port = COM3
monitor_speed = 115200
board_build.partitions = huge_app.csv
lib_deps =
    marcoschwartz/LiquidCrystal_I2C@^1.1.4
    madhephaestus/ESP32Servo@^3.0.5
    bblanchon/ArduinoJson@^7.0.4
```

- `upload_port`: porta USB onde a placa aparece no Windows (varia de PC para PC).
- `monitor_speed`: velocidade do **Serial Monitor**, onde a placa escreve mensagens.
- `huge_app.csv`: reserva mais espaço para o programa (a parte de internet segura, HTTPS, deixa o firmware grande).
- `lib_deps`: bibliotecas do LCD, dos servos e de leitura de JSON (usada no Telegram).

Para gravar: ✓ (Build) → → (Upload) → 🖱️ (Serial Monitor), na barra azul do rodapé do VSCode.

A4. Cinco conceitos antes de começar

1. **setup() e loop()** — todo programa Arduino tem essas duas funções. O `setup()` roda **uma vez** quando a placa liga; o `loop()` roda **para sempre**, milhares de vezes por segundo.
2. **millis() em vez de delay()** — `delay(1000)` congela a placa por 1 segundo (ela não lê o botão nem atende a página). Por isso o Zelo+ quase sempre usa `millis()`, que devolve "há quantos milissegundos a placa está ligada", e compara tempos: `cpp if (millis() - ultimaAtualizacao < 1000) return;` (se ainda não passou 1 segundo desde a última atualização, sai e tenta de novo na próxima volta do `loop()`, sem travar nada).
3. **Máquina de estados** — o dispenser está sempre em **um** estado (aguardando, tocando, porta aberta, abastecendo). Em cada estado ele faz coisas diferentes e, quando algo acontece, **muda de estado**. Isso deixa o comportamento previsível.
4. **Máscara de bits** — para dizer "compartimentos 1 e 3", o código usa um número em que cada **bit** representa um compartimento: `compartimento: 3 2 1 bits: 1 0 1 = 5` → compartimentos 1 e 3. Assim uma única variável (`uint8_t`) guarda qualquer combinação de compartimentos.
5. **Memória permanente (Preferences/NVS)** — a variável comum some quando a energia cai. A biblioteca `Preferences` grava dados na **memória flash** da placa, que não

se apaga ao desligar (como um pen drive).

Parte B — O código, passo a passo

Passo 1 — Bibliotecas e pinos

O que faz: carrega as bibliotecas e dá nome aos pinos, para o resto do código falar em `BUTTON_PIN` em vez de "5".

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <WebServer.h>
#include <DNSServer.h>
#include <Preferences.h>
#include <time.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ESP32Servo.h>

#define BUZZER_PIN 15
#define LED_PIN 4
#define BUTTON_PIN 5
#define NUM_COMPARTIMENTOS 3
#define MAX_HORARIOS 6 // horarios por medicamento
#define MAX_FAMILIARES 5
#define MAX_HISTORICO 60
```

Biblioteca	Para quê
WiFi, WebServer, DNSServer	Conectar à internet e servir a página de cadastro
WiFiClientSecure, HTTPClient, ArduinoJson	Falar com o Telegram (HTTPS + JSON)
Preferences	Memória permanente
time.h	Relógio (hora certa pela internet)
Wire, LiquidCrystal_I2C	LCD pelo barramento I2C
ESP32Servo	Controlar os servos

Os pinos e ângulos de cada servo ficam em **vetores** (listas), um item por compartimento:

```
const int PINOS_SERVO[NUM_COMPARTIMENTOS] = {13, 14, 27};
// Angulos de cada porta (ajuste aqui se algum mecanismo precisar de outro valor).
const int ANGULO_FECHADO[NUM_COMPARTIMENTOS] = {0, 0, 0};
const int ANGULO_ABERTO[NUM_COMPARTIMENTOS] = {90, 90, 90};
```

💡 `PINOS_SERVO[0]` é o compartimento **1** (em programação, a contagem começa em zero). Esse detalhe aparece no código inteiro: índice `c` = compartimento `c + 1`.

Por fim, os **objetos** que representam cada peça:

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo servos[NUM_COMPARTIMENTOS];
WebServer server(80);
DNSServer dnsServer;
Preferences preferences;
Preferences prefsCadastro;
Preferences prefsHistorico;
```

- `lcd(0x27, 16, 2)` : LCD no endereço I2C `0x27` , 16 colunas, 2 linhas.
- `server(80)` : servidor web na porta 80 (a porta padrão dos navegadores).
- Três `Preferences` , uma para cada "gaveta" da memória: Wi-Fi, cadastro e histórico.

Passo 2 — Estruturas de dados

O que faz: define como o programa guarda um **contato** e um **medicamento**.

```
struct Contato {
    String nome;
    String telefone;
    String chatId;
};

// Medicamento guardado num compartimento fixo (indice 0..2 = compartimento 1..3).
struct Medicamento {
    String nome;
    int totalHorarios = 0;
    int hora[MAX_HORARIOS];
    int minuto[MAX_HORARIOS];
    bool jaDisparado[MAX_HORARIOS];
};

String nomePaciente = "";
Medicamento medicamentos[NUM_COMPARTIMENTOS];
Contato cuidador;
Contato familiares[MAX_FAMILIARES];
```

- `struct` agrupa várias informações sob um nome só — como uma ficha.
- `medicamentos[3]` : a **posição no vetor é o compartimento**. O remédio em `medicamentos[1]` está sempre no compartimento 2. É isso que "prende" cada remédio ao seu compartimento.
- `jaDisparado[]` : impede que o mesmo horário toque duas vezes no mesmo minuto.
- `chatId` : número que o Telegram usa para identificar cada pessoa (Passo 15).

Os **estados** da máquina de estados:

```
enum Estado { AGUARDANDO, TOCANDO, PORTA_ABERTA_ESTADO, ABASTECENDO };
Estado estadoAtual = AGUARDANDO;
```

E as variáveis da **dose atual** (usam máscara de bits, conceito 4):

```
uint8_t doseMascara = 0;
String horarioDose = "";
String nomesDose = "";
unsigned long alarmeIniciadoEm = 0;
int nivelAvisoEnviado = 0; // 0 = nenhum, 1 = cuidador avisado, 2 = todos alertados
```

```
bool dosePendente = false; // alarme terminou sem acesso; botao ainda abre os
compartimentos
```

Duas funções pequenas ajudam a trabalhar com isso:

```
bool temBit(uint8_t mascara, int i) {
    return (mascara >> i) & 1;
}

bool medicamentoAtivo(int c) {
    return medicamentos[c].nome != "" && medicamentos[c].totalHorarios > 0;
}
```

- `temBit(5, 0)` → verdadeiro (compartimento 1 está na máscara 101).
- `medicamentoAtivo(c)`: um compartimento só conta se tiver nome e horário.

Passo 3 — `setup()`: ligando tudo

O que faz: roda uma vez ao ligar. Configura pinos, posiciona as portas, liga o LCD, lê a memória e tenta conectar no Wi-Fi.

```
pinMode(BUZZER_PIN, OUTPUT);
pinMode(LED_PIN, OUTPUT);
pinMode(BUTTON_PIN, INPUT_PULLUP);
digitalWrite(BUZZER_PIN, LOW);
digitalWrite(LED_PIN, LOW);

for (int c = 0; c < NUM_COMPARTIMENTOS; c++) {
    servos[c].attach(PINOS_SERVO[c]);
    servos[c].write(ANGULO_FECHADO[c]);
}
```

- `OUTPUT`: o ESP32 **manda** energia no pino (buzzer, LED).
- `INPUT_PULLUP`: o pino fica "puxado" para 3,3 V por um resistor **interno**. Quando o botão é apertado, ele liga o pino ao GND e a leitura vira `LOW`. Por isso o botão não precisa de resistor externo, e "apertado" é `digitalRead(BUTTON_PIN) == LOW`.
- O `for` percorre os 3 servos, liga cada um no seu pino e fecha a porta.

Depois, carrega a memória e decide o modo de Wi-Fi:

```
prefsCadastro.begin("cadastro", false);
carregarCadastro();
prefsHistorico.begin("historico", false);
carregarHistorico();

preferences.begin("wifi", false);
String ssidSalvo = preferences.getString("ssid", "");
String senhaSalva = preferences.getString("pass", "");
```

```
if (WiFi.status() == WL_CONNECTED) {
    iniciarModoNormal();
} else {
    iniciarModoConfig();
}
```

- Se existe rede salva e a conexão funciona → **modo normal** (Passo 5).
- Se não → **modo de configuração** (Passo 4).

Passo 4 — Wi-Fi sem senha no código (captive portal)

O que faz: quando a placa não sabe em qual Wi-Fi entrar, ela **cria a sua própria rede** chamada `ZeloPlus-Config`. Ao conectar o celular nela, a página de configuração abre sozinha (igual ao Wi-Fi de aeroporto).

```
void iniciarModoConfig() {
    modoConfig = true;

    WiFi.mode(WIFI_AP_STA);
    WiFi.softAPConfig(apIP, apIP, IPAddress(255, 255, 255, 0));
    WiFi.softAP("ZeloPlus-Config");

    dnsServer.start(DNS_PORT, "*", apIP);
```

- `softAP` : transforma o ESP32 em um **ponto de acesso** (roteador).
- `dnsServer.start(..., "*", apIP)` : responde **qualquer** endereço digitado com o IP da placa (`192.168.4.1`). É esse "truque" que faz o celular abrir a página sozinho.

Quando o cuidador escolhe a rede e digita a senha, a placa testa e, se der certo, **grava** e reinicia:

```
if (WiFi.status() == WL_CONNECTED) {
    preferences.putString("ssid", ssid);
    preferences.putString("pass", senha);
```

```
    delay(3000);
    ESP.restart();
```

Da próxima vez que ligar, o `setup()` encontra a rede salva e vai direto para o modo normal. O link "Trocar rede Wi-Fi" da página apaga essa rede (`preferences.remove("ssid")`) e reinicia no modo de configuração.

Passo 5 — Modo normal: relógio pela internet e rotas da página

O que faz: acerta o relógio pela internet, mostra o IP no LCD e diz ao servidor qual função atende cada endereço da página.

```
const char* ntpServer = "pool.ntp.org";
const long  gmtOffset_sec = -10800; // Brasília (GMT-3)
const int  daylightOffset_sec = 0;
```

```
configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
```

- **NTP** é um serviço da internet que informa a hora exata. `-10800` segundos = 3 horas a menos que o horário mundial (UTC) = horário de Brasília.

As **rotas** ligam um endereço a uma função:

```
server.on("/", handleRoot);
server.on("/salvar", HTTP_POST, handleSalvar);
server.on("/abrir-manual", HTTP_POST, handleAbrirManual);
server.on("/esvaziar", HTTP_POST, handleEsvaziar);
server.on("/testar-mensagens", HTTP_POST, handleTestarMensagens);
server.on("/telegram-ids", HTTP_POST, handleTelegramIds);
server.on("/historico.csv", handleHistoricoCSV);
server.on("/limpar-historico", HTTP_POST, handleLimparHistorico);
server.on("/trocar-wifi", handleTrocarWifi);
server.begin();
```

Endereço	O que acontece
/	Mostra a página de cadastro (Passo 7)
/salvar	Recebe o formulário e grava (Passo 8)
/abrir-manual	Abre um compartimento para reposição (Passo 14)
/esvaziar	Esvazia um compartimento (Passo 14)
/testar-mensagens , /telegram-ids	Telegram (Passo 15)
/historico.csv , /limpar-historico	Histórico (Passo 16)

Passo 6 — Memória permanente (Preferences)

O que faz: grava e lê o cadastro na memória flash, para ele sobreviver a quedas de energia.

Cada compartimento tem suas chaves: `m0_` (compartimento 1), `m1_`, `m2_`.

```
void salvarMedicamento(int c) {
    const Medicamento& m = medicamentos[c];
    String p = prefixoMedicamento(c);
    uint8_t horas[MAX_HORARIOS];
    uint8_t minutos[MAX_HORARIOS];
    for (int i = 0; i < m.totalHorarios; i++) {
        horas[i] = (uint8_t)m.hora[i];
        minutos[i] = (uint8_t)m.minuto[i];
    }
    prefsCadastro.putString((p + "_nome").c_str(), m.nome);
    prefsCadastro.putUChar((p + "_tot").c_str(), (uint8_t)m.totalHorarios);
    if (m.totalHorarios > 0) {
        prefsCadastro.putBytes((p + "_hr").c_str(), horas, m.totalHorarios);
        prefsCadastro.putBytes((p + "_mn").c_str(), minutos, m.totalHorarios);
    }
}
```

- `putString` grava texto; `putUChar` grava um número pequeno (0-255); `putBytes` grava uma lista de bytes (as horas e os minutos).
- Exemplo do que fica gravado para "Losartana às 08:00 e 20:00" no compartimento 1:
`m0_nome = "Losartana" , m0_tot = 2 , m0_hr = [8, 20] , m0_mn = [0, 0] .`

Na leitura, o programa **confere** se os dados fazem sentido antes de usar:

```
int validos = 0;
for (int i = 0; i < total; i++) {
    if (horas[i] > 23 || minutos[i] > 59) continue;
    horaDestino[validos] = horas[i];
    minutoDestino[validos] = minutos[i];
    disparadoDestino[validos] = false;
    validos++;
}
return validos;
```

Migração: a versão anterior (um compartimento só) gravava o remédio em chaves diferentes. Na primeira vez que a versão nova liga, ela copia esse remédio para o compartimento 1 e apaga as chaves antigas:

```
void migrarCadastroAntigo() {
    if (prefsCadastro.containsKey("m0_nome") || !prefsCadastro.containsKey("remedio")) return;

    Medicamento m = medicamentos[0];
    m.nome = prefsCadastro.getString("remedio", "");
    m.totalHorarios = lerHorarios("total", "horas", "minutos", m.hora, m.minuto,
    m.jaDisparado);
    salvarMedicamento(0);
}
```

Passo 7 — A página de cadastro

O que faz: quando o navegador abre o IP da placa, a função `handleRoot()` **monta o HTML** (o texto da página) e envia. Cada compartimento vira um cartão.

```
html += "<h3 style='margin:18px 0 0 0'>Medicamentos</h3>";
html += "<p style='font-size:13px;color:#6b7280;margin:4px 0'>Cada medicamento fica sempre no mesmo compartimento. Deixe em branco os compartimentos que nao forem usados.</p>";
for (int c = 0; c < NUM_COMPARTIMENTOS; c++) {
    html += cartaoCompartimento(c);
}
```

Dentro do cartão, a parte mais importante é a **dica de onde colocar o remédio**:

```
html += "&#128230; Coloque <b>" + escaparHTML(m.nome) + "</b> no <b>compartimento " + n + " </b></p>";
```

Resultado na página: 📦 Coloque **Losartana** no **compartimento 1**.

A dica muda **enquanto o cuidador digita**, com um pequeno código JavaScript que roda no navegador:

```
html += "function atualizarDica(c) {";
html += "    var nome = document.getElementById('m' + c + '_nome').value.trim();";
html += "    var dica = document.getElementById('dica-' + c);";
html += "    dica.style.display = nome ? 'block' : 'none';";
```

Troca de remédio num compartimento já usado — antes de enviar, o navegador compara o nome antigo (guardado em `data-original`) com o novo e pede confirmação:

```
html += "    if (antigo && novo && antigo.toLowerCase() != novo.toLowerCase()) {";
html += "    if (!confirm('O compartimento ' + (c + 1) + ' tinha ' + antigo + '. Retire todos os comprimidos antigos antes de colocar ' + novo + '. Confirmar troca?')) return false;";
```

 **Segurança:** todo texto digitado passa por `escaparHTML()` antes de entrar na página. Assim, um nome como `D'Ávila` ou `<b>` não quebra o HTML.

```
cpp case '<': saida += "&lt;"; break;
```

Passo 8 — Salvando o cadastro

O que faz: `handleSalvar()` recebe o formulário, **valida** tudo e só então grava.

1. Cuidador é obrigatório:

```
if (novoCuidador.nome == "" || novoCuidador.telefone == "") {
    server.send(400, "text/plain; charset=utf-8", "O cuidador é obrigatório: preencha nome e celular.");
    return;
}
```

1. Cada remédio precisa de horário, e ao menos um remédio precisa existir:

```
if (novos[c].nome != "" && novos[c].totalHorarios == 0) {
```

```
if (ativos == 0) {
    server.send(400, "text/plain; charset=utf-8", "Cadastre ao menos um medicamento. Volte e corrija.");
    return;
}
```

1. Descubra quais compartimentos receberam remédio novo e os coloca numa **fila de abastecimento** (máscara de bits):

```
if (novo != "" && novo != antigo) filaAbastecimento |= (uint8_t)(1 << c);
if (novo == "") filaAbastecimento &= (uint8_t)~(1 << c);
medicamentos[c] = novos[c];
```

- `|= (1 << c)` **liga** o bit do compartimento `c` ("este precisa abrir").
- `&= ~(1 << c)` **desliga** o bit.
- Se só os horários mudaram, `novo == antigo` e nada entra na fila → a porta não abre.
- **Grava tudo** e devolve o navegador para a página:

```
salvarCadastro();

server.sendHeader("Location", "/");
server.send(303);
```

A abertura das portas **não** acontece aqui: quem cuida disso é o `loop()` (Passo 10), um compartimento de cada vez.

Passo 9 — Movendo as portas (servos)

O que faz: abre e fecha as portas **devagar** (1 grau a cada 15 ms), para não dar tranco no mecanismo.

```
void abrirCompartimento(int c) {
    for (int angulo = ANGULO_FECHADO[c]; angulo <= ANGULO_ABERTO[c]; angulo++) {
        servos[c].write(angulo);
        delay(15);
    }
}
```

- 90 passos × 15 ms ≈ **1,4 s** para abrir.

Para abrir **vários** compartimentos com um único toque, sem que dois servos se movam juntos (o que sobrecarregaria a fonte), eles abrem **um depois do outro**:

```
// Abre os compartimentos marcados, um logo apos o outro.
void abrirCompartimentos(uint8_t mascara) {
    for (int c = 0; c < NUM_COMPARTIMENTOS; c++) {
        if (temBit(mascara, c)) abrirCompartimento(c);
    }
}
```

Como `abrirCompartimento()` só termina depois que a porta chegou em 90°, o próximo servo só começa quando o anterior parou.

Passo 10 — O coração: `loop()` e a máquina de estados

O que faz: a cada volta, atende a página web, confere os horários e executa o que o **estado atual** manda.

```
void loop() {
    if (modoConfig) {
        dnsServer.processNextRequest();
    }
    server.handleClient();

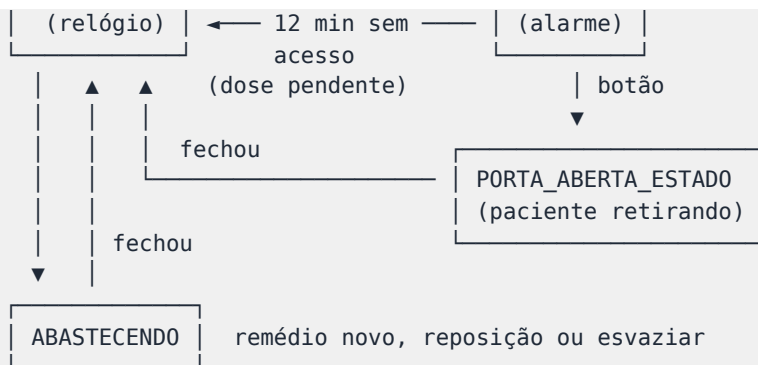
    if (modoConfig) {
        return;
    }

    verificarHorarios();

    switch (estadoAtual) {
```

Mapa dos estados:





No estado **AGUARDANDO**, a ordem de prioridade é:

```

case AGUARDANDO:
    if (dosePendente && digitalRead(BUTTON_PIN) == LOW) {
        abrirParaPaciente();
        break;
    }
    // Prioridade: alarme em espera; depois abastecimento de compartimento novo.
    if (mascaraEmEspera != 0) {
        uint8_t mascara = mascaraEmEspera;
        mascaraEmEspera = 0;
        iniciarAlarme(mascara, horarioEmEspera);
        break;
    }
    if (filaAbastecimento != 0) {

```

1. Dose pendente + botão → abre para o paciente.
2. Algum horário esperando → começa o alarme.
3. Algum compartimento novo na fila → abre para abastecer.
4. Nada disso → mostra o relógio.

Passo 11 — Conferindo os horários

O que faz: uma vez por segundo, compara a hora atual com os horários de cada remédio. Funciona **em qualquer estado**.

```

for (int c = 0; c < NUM_COMPARTIMENTOS; c++) {
    if (!medicamentoAtivo(c)) continue;
    Medicamento& m = medicamentos[c];
    for (int i = 0; i < m.totalHorarios; i++) {
        if (m.jaDisparado[i] || timeinfo.tm_hour != m.hora[i] || timeinfo.tm_min !=
m.minuto[i]) continue;
        m.jaDisparado[i] = true;
        if (mascaraEmEspera == 0) horarioEmEspera = formatarHorario(m.hora[i], m.minuto[i]);
        mascaraEmEspera |= (uint8_t)(1 << c);
    }
}

```

- Quando bate o horário, o compartimento entra na **máscara de espera**.
- **Dois remédios no mesmo horário** entram na mesma máscara → viram **um único alarme**.

- Se o dispenser estiver ocupado (ex.: cuidador repondo outro compartimento), o horário **fica esperando** e toca assim que ele voltar a **AGUARDANDO**.
- `jaDisparado` impede de tocar de novo no mesmo minuto; ele é "rearmado" quando o minuto muda:

```
if (timeinfo.tm_min != medicamentos[c].minuto[i]) medicamentos[c].jaDisparado[i] = false;
```

Passo 12 — O alarme tocando

O que faz: `iniciarAlarme()` prepara a dose e muda para **TOCANDO**.

```
void iniciarAlarme(uint8_t mascara, const String& horario) {
    estadoAtual = TOCANDO;
    doseMascara = mascara;
    horarioDose = horario;
    nomesDose = juntarNomes(mascara);
    alarmeIniciadoEm = millis();
    nivelAvisoEnviado = 0;
    dosePendente = false;
```

- `nomesDose` guarda o texto "Losartana e Metformina" para as mensagens.
- `alarmeIniciadoEm` marca o instante do início: todos os tempos são contados a partir dele.

Ciclo do som — 1 minuto tocando, 1 minuto em silêncio:

```
bool minutoDeSom = ((decorrido / CICLO_ALARME) % 2) == 0;
if (!minutoDeSom) {
    if (ledBuzzerLigado) desligarLedBuzzer();
    break;
}

if (millis() - ultimoToggleAlarme >= 150) {
    ultimoToggleAlarme = millis();
    ledBuzzerLigado = !ledBuzzerLigado;
    digitalWrite(LED_PIN, ledBuzzerLigado ? HIGH : LOW);
    digitalWrite(BUZZER_PIN, ledBuzzerLigado ? HIGH : LOW);
}
```

- `decorrido / CICLO_ALARME` = quantos minutos se passaram (0, 1, 2, ...).
- `% 2 == 0` → minutos pares (0, 2, 4...) tocam; ímpares ficam em silêncio.
- Dentro do minuto de som, LED e buzzer **invertem a cada 150 ms** (bip-bip-bip).

Avisos — aos 6 minutos para o cuidador, aos 12 para todos:

```
if (decorrido >= TEMPO_PRIMEIRO_AVISO && nivelAvisoEnviado == 0) {
    desligarLedBuzzer();
    escreverLinhaLCD(1, "Avisando cuidad.");
    avisarPrimeiroAtraso();
    nivelAvisoEnviado = 1;
}
```

```
if (decorrido >= TEMPO_ALERTA_FINAL) {
    desligarLedBuzzer();
```

```

lcd.clear();
escreverLinhaLCD(0, "Avisando familia");
atualizarDose(DOSE_SEM_ACESSO, decorrido);
alertarSemAcesso();
nivelAvisoEnviado = 2;

```

```

dosePendente = true;
lcd.clear();
estadoAtual = AGUARDANDO;

```

- `nivelAvisoEnviado` evita mandar o mesmo aviso duas vezes.
- Depois dos 12 minutos o alarme **para**, mas a dose fica **pendente**: se o paciente aparecer mais tarde, o botão ainda abre os compartimentos.

Passo 13 — O paciente aperta o botão

O que faz: registra o acesso, desliga o alarme e **abre todos os compartimentos da dose**, um após o outro.

```

void abrirParaPaciente() {
    unsigned long decorrido = millis() - alarmeIniciadoEm;
    atualizarDose(decorrido < TEMPO_PRIMEIRO_AVISO ? DOSE_NO_HORARIO : DOSE_ATRASADA,
decorrido);
    esquecerRegistrosDose();

    desligarLedBuzzer();
    dosePendente = false;

    lcd.clear();
    escreverLinhaLCD(0, "Retire: " + listaCompartimentos(doseMascara));
    escreverLinhaLCD(1, "Abrindo...");
    abrirCompartimentos(doseMascara);

    mascaraAberta = doseMascara;
    estadoAtual = PORTA_ABERTA_ESTADO;
    portaAbertaEm = millis();

    avisarAcessoAposAviso();
}

```

- Antes do 1º aviso → "tomada no horário"; depois → "tomada com atraso" (Passo 16).
- Se já tinha avisado alguém, `avisarAcessoAposAviso()` manda "Paulo acessou o dispenser às 15:27", para a família ficar tranquila.

Fechando — trava de 7 segundos: logo depois de abrir, o dedo do paciente ainda pode estar no botão. Para não fechar sem querer, o botão só vale depois de 7 s. Se ninguém apertar, fecha sozinho após 3 minutos:

```

bool travaLiberada = (millis() - portaAbertaEm >= TRAVA_BOTA0);
bool fechar = (travaLiberada && digitalRead(BUTTON_PIN) == LOW) ||
(millis() - portaAbertaEm >= TEMPO_PORTA_ABERTA);

if (fechar) {
    lcd.clear();
    escreverLinhaLCD(0, "Fechando porta..");
}

```

```
fecharCompartimentos(mascaraAberta);
mascaraAberta = 0;
```

Passo 14 — Abastecer, repor e esvaziar

Os três casos usam a **mesma função**, que abre **um** compartimento para o cuidador e só muda o texto do LCD:

```
void iniciarAbastecimento(int c, ModoAbastecimento modo) {
    compartimentoAbastecendo = c;
    modoAbastecimento = modo;

    String titulo;
    if (modo == ABAST_REPOR) titulo = "Repor compart." + String(c + 1);
    else if (modo == ABAST_ESVAZIAR) titulo = "Esvaziar comp." + String(c + 1);
    else titulo = "Abast. compart." + String(c + 1);
```

a) Abastecimento guiado (remédio novo): o `loop()` pega o **primeiro** compartimento da fila, tira ele da fila e abre. Quando a porta fecha, o estado volta para `AGUARDANDO` e o próximo da fila abre — um de cada vez:

```
for (int c = 0; c < NUM_COMPARTIMENTOS; c++) {
    if (!temBit(filaAbastecimento, c)) continue;
    filaAbastecimento &= (uint8_t)~(1 << c);
    iniciarAbastecimento(c, ABAST_NOVO);
    break;
}
```

b) Reposição: na página, o cuidador **escolhe o remédio** (os botões de opção só mostram os cadastrados), confirma, e o navegador envia o número do compartimento:

```
html += "  var corpo = new URLSearchParams(); corpo.append('c', reporEscolhido);";
html += "  fetch('/abrir-manual', {method:'POST', body: corpo}).then(function(r){";
```

A placa confere se o pedido é válido e se está livre, e abre **só aquele**:

```
void handleAbrirManual() {
    int c = server.arg("c").toInt();
    if (!server.hasArg("c") || c < 0 || c >= NUM_COMPARTIMENTOS || !medicamentoAtivo(c)) {
        server.send(400, "text/plain", "Compartimento invalido");
        return;
    }
    if (!dispenserLivre()) {
        server.send(409, "text/plain", "Sistema ocupado no momento");
        return;
    }

    server.send(200, "text/plain", "OK");
    iniciarAbastecimento(c, ABAST_REPOR);
}
```

c) Esvaziar: apaga o remédio do compartimento e, se o cuidador pediu, abre para retirar as sobras:

```

medicamentos[c].nome = "";
medicamentos[c].totalHorarios = 0;
filaAbastecimento &= (uint8_t)~(1 << c);
salvarMedicamento(c);

```

Passo 15 — Avisos pelo Telegram

Por que Telegram? É gratuito, oficial e funciona com uma simples requisição pela internet. (O WhatsApp gratuito via CallMeBot ficou sem vagas; o WhatsApp oficial é pago e exige aprovação da Meta.)

Configuração (uma vez): o cuidador cria um bot no **@BotFather**, cola o **token** na página; cada pessoa abre o bot e toca em **Iniciar**; o botão "Buscar IDs" mostra o **chatId** de cada uma.

Enviar uma mensagem é fazer um pedido HTTPS para a API do Telegram:

```

String url = "https://api.telegram.org/bot" + token + "/" + metodo;
if (!http.begin(cliente, url)) {
    Serial.println("Telegram: falha ao iniciar conexao");
    return -1;
}
http.addHeader("Content-Type", "application/x-www-form-urlencoded");
int codigo = http.POST(corpo);

```

```

String corpo = "chat_id=" + codificarURL(contato.chatId) + "&text=" + codificarURL(texto);
int codigo = chamarTelegram(tokenTelegram, "sendMessage", corpo);

```

- **codificarURL()** troca espaços e acentos por códigos (`%20` , `%C3%A3` ...), porque o endereço/corpo do pedido só aceita alguns caracteres.
- A resposta `200` significa "entregue ao Telegram".

As mensagens:

```

void avisarPrimeiroAtraso() {
    enviarParaCuidador("Zelo+: Paciente " + nomePaciente + " não acessou o medicamento das "
+
        horarioDose + " horas (" + nomesDose + ").");
}

void alertarSemAcesso() {
    enviarParaTodos("Zelo+: Paciente " + nomePaciente + " não foi até o dispenser no horário
das " +
        horarioDose + " (" + nomesDose + ").");
}

```

Momento	Quem recebe	Exemplo
6 min sem acesso	Cuidador	Zelo+: Paciente "Paulo" não acessou o medicamento das "15:26" horas (Losartana).
12 min sem acesso	Cuidador + familiares	Zelo+: Paciente "Paulo" não foi até o dispenser no horário das "15:26" (Losartana e Metformina).
Acesso depois de um aviso	Quem foi avisado	Zelo+: Paulo acessou o dispenser às 15:27 (dose de Losartana das 15:26).

Buscar IDs: a função `handleTelegramIds()` pede ao Telegram as últimas mensagens recebidas pelo bot (`getUpdates`) e usa a **ArduinoJson** para ler só o que interessa (id e nome de cada pessoa):

```
JsonDocument filtro;
JsonObject chatFiltro = filtro["result"][0]["message"]["chat"].to<JsonObject>();
chatFiltro["id"] = true;
chatFiltro["first_name"] = true;
chatFiltro["last_name"] = true;
chatFiltro["username"] = true;
```

O **filtro** economiza memória: a resposta do Telegram tem muitos campos, mas a placa guarda apenas esses quatro.

Passo 16 — Histórico de doses

O que faz: cada remédio de cada alarme vira um **registro** com data, situação e atraso. Serve para calcular a **adesão ao tratamento**.

```
struct RegistroDose {
    uint32_t inicio;           // horario do disparo (segundos desde 1970)
    uint16_t atrasoSeg;       // tempo ate o acesso
    uint8_t status;
    uint8_t compartimento;    // 0..2
    char remedio[20];         // nome do medicamento no momento da dose
};
```

```
enum StatusDose : uint8_t {
    DOSE_PENDENTE = 0,       // alarme tocando (ou placa reiniciou antes de concluir)
    DOSE_NO_HORARIO = 1,    // acesso antes do primeiro aviso ao cuidador
    DOSE_ATRASADA = 2,      // acesso depois do primeiro aviso (inclusive apos o alerta final)
    DOSE_SEM_ACESSO = 3     // alerta final enviado e ninguem acessou
};
```

Buffer circular: a placa guarda as **60 doses mais recentes**. Quando enche, a próxima sobrescreve a mais antiga — como uma fila que dá a volta:

```
int registrarInicioDose(int compartimento) {
    int idx = proximoHistorico;
```

```
    proximoHistorico = (proximoHistorico + 1) % MAX_HISTORICO;
    if (totalHistorico < MAX_HISTORICO) totalHistorico++;
```

```
return idx;
}
```

- `% MAX_HISTORICO` (resto da divisão) faz a posição voltar a 0 depois da 59.

Para ler do mais recente para o mais antigo:

```
int indiceHistorico(int k) {
    return (proximoHistorico - 1 - k + 2 * MAX_HISTORICO) % MAX_HISTORICO;
}
```

Adesão = doses tomadas (no horário + com atraso) ÷ doses concluídas, nos últimos 7 dias. O arredondamento é feito somando metade do divisor:

```
int percentual(int parte, int total) {
    return (parte * 100 + total / 2) / total;
}
```

A página mostra a adesão geral e por remédio, a tabela das 20 doses mais recentes e o link **Baixar histórico (CSV)**, que abre no Excel/Google Planilhas:

```
String csv = "data;horario;medicamento;compartimento;situacao;atraso_minutos\r\n";
```

Passo 17 — Escrevendo no LCD

O problema: o LCD 16x2 não tem acentos. "Potássica" apareceria com símbolos estranhos.

A solução: converter para maiúsculas sem acento.

Em UTF-8 (o formato do texto), letras acentuadas usam **2 bytes**, e o primeiro é sempre `0xC3`. O segundo byte diz qual é a letra:

```
if (c == 0xC3 && i + 1 < texto.length()) {
    uint8_t d = (uint8_t)texto[++i] | 0x20; // 0x80-0x9F (maiusculas) -> minusculas
    char base = '?';
    if (d >= 0xA0 && d <= 0xA5) base = 'A';
    else if (d == 0xA7) base = 'C';
    else if (d >= 0xA8 && d <= 0xAB) base = 'E';
}
```

Resultado: "Losartana Potássica" → "LOSARTANA POTASS" (cortado em 16 colunas).

E uma função que **sempre preenche a linha inteira**, apagando restos de textos anteriores:

```
void escreverLinhaLCD(int linha, const String& texto) {
    String t = textoLCD(texto);
    while (t.length() < 16) t += ' ';
    lcd.setCursor(0, linha);
    lcd.print(t);
}
```

Situação	Linha 1	Linha 2
Aguardando	15:26:03	3 remédios
Alarme	Hora do remédio!	LOSARTANA (C1) (alterna a cada 2 s)
Retirada	Retire: C1 C2	Fecha em: 170s
Abastecer / Repor	Abast. compart.2 / Repor compart.2	METFORMINA
Dose pendente	15:40:10	Dose pendente!

Passo 18 — Modo de teste

Esperar 12 minutos a cada teste seria lento. Uma única linha troca todos os tempos:

```
#define MODO_TESTE 1

#if MODO_TESTE
const unsigned long CICLO_ALARME = 5UL * 1000UL;
const unsigned long TEMPO_PRIMEIRO_AVISO = 30UL * 1000UL;
const unsigned long TEMPO_ALERTA_FINAL = 60UL * 1000UL;
#else
const unsigned long CICLO_ALARME = 60UL * 1000UL;
const unsigned long TEMPO_PRIMEIRO_AVISO = 6UL * 60UL * 1000UL;
const unsigned long TEMPO_ALERTA_FINAL = 12UL * 60UL * 1000UL;
#endif
```

- `#if` é resolvido **na compilação**: só um dos blocos vai para a placa.
- Com `MODO_TESTE 1`: aviso aos 30 s, alerta aos 60 s, som 5 s / silêncio 5 s.
- **Antes do uso real**, trocar para `#define MODO_TESTE 0` e gravar de novo.
- O `UL` ("unsigned long") evita que a conta estoure: `12 * 60 * 1000` passa do limite de um `int` em algumas placas.

Parte C — Juntando tudo

C1. Linha do tempo de uma dose

Exemplo: **Losartana (compartimento 1)** e **Metformina (compartimento 2)** às **15:26**, modo real.

Hora	O que acontece	Onde no código
15:26:00	<code>verificarHorarios()</code> coloca C1 e C2 na máscara de espera	Passo 11
15:26:00	<code>loop()</code> em <code>AGUARDANDO</code> chama <code>iniciarAlarme(0b011, "15:26")</code> ; cria 2 registros "tocando"	Passos 10, 12, 16
15:26–15:27	Buzzer e LED bip-bip; LCD alterna <code>LOSARTANA (C1)</code> / <code>METFORMINA (C2)</code>	Passos 12, 17
15:27–15:28	Silêncio (botão continua valendo)	Passo 12
15:32	Sem acesso → Telegram para o cuidador	Passos 12, 15
15:33	Paciente aperta o botão → registros "com atraso (7 min)"; abre C1, depois C2	Passo 13
15:33	Telegram "Paulo acessou o dispenser às 15:33" para o cuidador	Passo 15
15:33:07	Trava liberada; LCD <code>Fecha em: 173s</code>	Passo 13
15:34	Paciente aperta → fecha C1, depois C2 → <code>AGUARDANDO</code>	Passo 13

Se o paciente **não** aparecer até 15:38 (12 min): alerta para todos, registros "sem acesso", LCD `Dose pendente!`, e o botão ainda abre os dois compartimentos depois.

C2. Referência rápida

Tempos

Constante	Modo real	Modo teste
<code>CICLO_ALARME</code>	1 min	5 s
<code>TEMPO_PRIMEIRO_AVISO</code>	6 min	30 s
<code>TEMPO_ALERTA_FINAL</code>	12 min	60 s
<code>TRAVA_BOTAO</code>	7 s	7 s
<code>TEMPO_PORTA_ABERTA</code> / <code>TEMPO_ABASTECIMENTO</code>	3 min	3 min

Memória (Preferences)

Gaveta (namespace)	Chaves
wifi	ssid, pass
cadastro	nome (paciente), m0_* / m1_* / m2_* (remédios), cuid_*, fam0_* ... fam4_*, tg_token
historico	regs2 (registros), total, prox

Limites

O quê	Máximo
Compartimentos/remédios	3
Horários por remédio	6
Familiares	5
Doses no histórico	60

C3. Para praticar

Sugestões de exercícios, do mais simples ao mais desafiador:

1. **Mudar o tempo da trava** de 7 s para 5 s (Passo 13: TRAVA_BOTA0).
2. **Trocar o bip**: fazer o buzzer inverter a cada 300 ms em vez de 150 ms (Passo 12).
3. **Mensagem no LCD**: mostrar Bom dia! na linha 2 entre 6h e 12h quando não houver dose pendente (Passo 17, atualizarLCDRelogio()).
4. **Ângulo diferente**: o compartimento 3 abre só até 70° (Passo 1: ANGULO_ABERTO).
5. **Quarto compartimento**: o que precisaria mudar? (Dica: NUM_COMPARTIMENTOS, PINOS_SERVO, os ângulos... e a página já se adapta sozinha.)
6. **Desafio**: enviar pelo Telegram um resumo diário de adesão às 21h (Passos 11, 15 e 16).